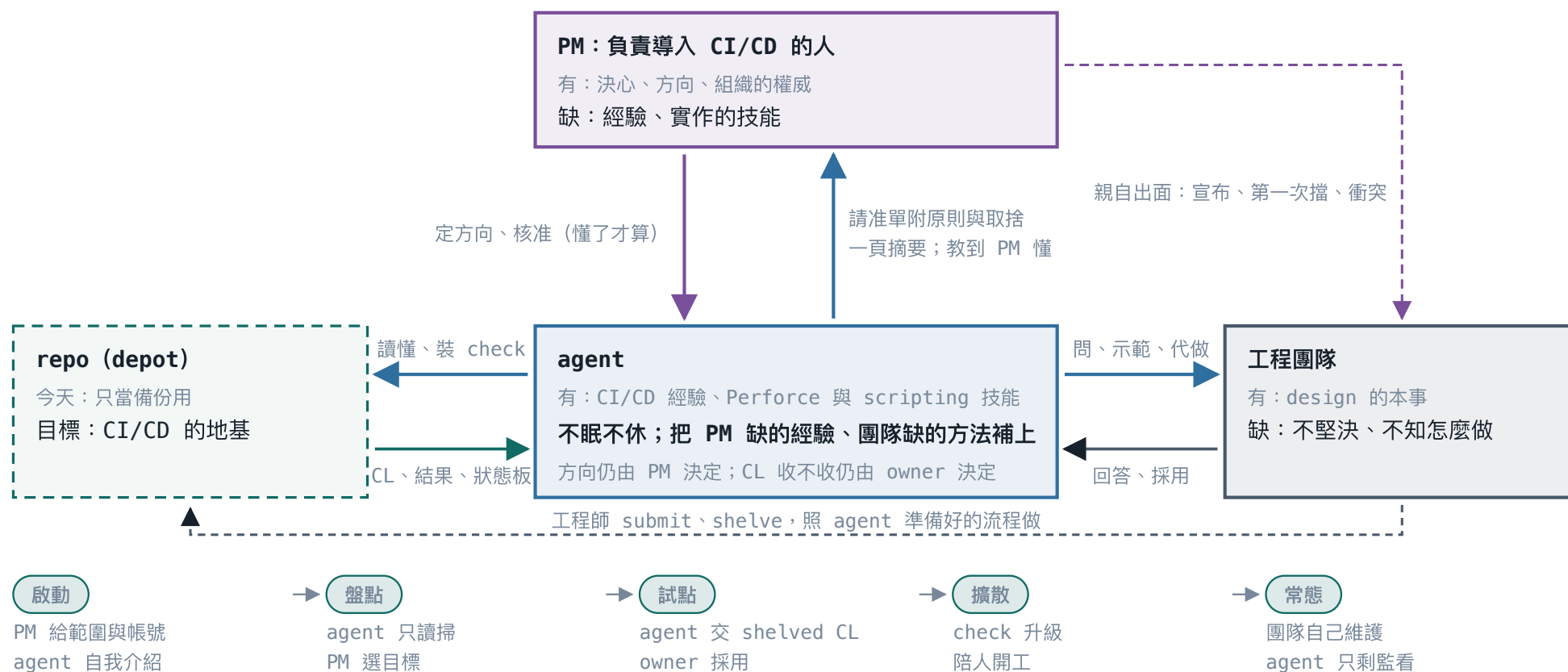


總覽：PM 有決心沒經驗，團隊不堅決也不知怎麼做；agent 補這兩個缺



PM 有決心沒經驗，團隊不堅決也不知道怎麼做；agent 補的正是這兩個缺：給 PM 可核准的提案與經驗，給團隊方法與陪做。

後面每頁一個場景；下一頁是每個階段誰做什麼的總表、授權三級、用語。

總表：每個階段誰做什麼、agent 的授權三級、用語定義

每個階段誰做什麼（上一頁的展開）；agent 自己能做什麼、什麼要問、什麼要 PM 決定 ↓

	啟動	盤點	目標分析與訪談	計畫核准	建置	上線分級	監看與開工	擴充與交棒
PM	給範圍、帳號與預算	選候選目標	看進度	懂了才核准	看進度	同意擋不擋	看一頁摘要	定交棒條件
agent	bot 上線 自我介紹	只讀掃 一頁現況	先讀懂 再問 owner	附取捨 請核准	做成 shelved CL	附證據 提議升級	看 CL、私訊 登記開工	提下一道 只剩監看
工程團隊	知道 agent 在 看得到什麼		owner 答只有 他知道的	owner 同意 動範圍	負責人 補強 script	同意升級	回一句 照流程開工	自己維護 pipeline
repo	唯讀；紀錄在 agent 的 repo	唯讀	乾淨 workspace 實跑	決定記在 agent 的 repo	shelved CL + 證據	報告→警告→擋	跑 check 狀態板	check 穩定 趨勢

agent 的授權三級

自主

 讀、分析、寫地圖、開 shelved CL、私訊

告知

 建議、第二次提醒、交 shelved CL

請准

 方向、新規範、裝 trigger、擋 submit、拉 PL 群聊

用語

PM=負責把團隊開發流程導入 CI/CD 的人（和 project 的 PM 無關） owner=目標目錄或模組的負責人，決定 agent 交的 CL 收不收

CL 作者=submit 那一包的工程師 PROJECT_MAP=agent 寫的目錄地圖，主本在它自己的 repo 狀態板=每個模組休止 / 進行中哪些任務的表

subagent=agent 分出去做事的子程式 shelved CL=給人看、還沒 submit 的改動 (git 的 PR)

agent 先讀懂再問，改動做成 shelved CL；方向與影響別人的事，PM 懂了才算核准；CL 收不收是 owner 的事。

後面每頁一個場景，四條泳道，箭頭就是誰對誰做什麼；最後兩頁是橫跨全程的規則。

啟動：PM 給 depot 範圍與帳號，agent 以 bot 身分先只讀上線，先向團隊自我介紹

什麼時候：PM 決定開始的第一天

授權：PM 請准



示意的自我介紹：「hi，我是 Eric，負責幫 dma 導入 CI/CD 的 AI agent，向 PM 某某報告。我會看 //depot/chipA/dma 的 CL 與 check 結果，紀錄在我自己的 git repo，你們有讀的權限。有問題直接私訊我。」（別名由 PM 挑定，顯示名稱帶 AI agent；D19）

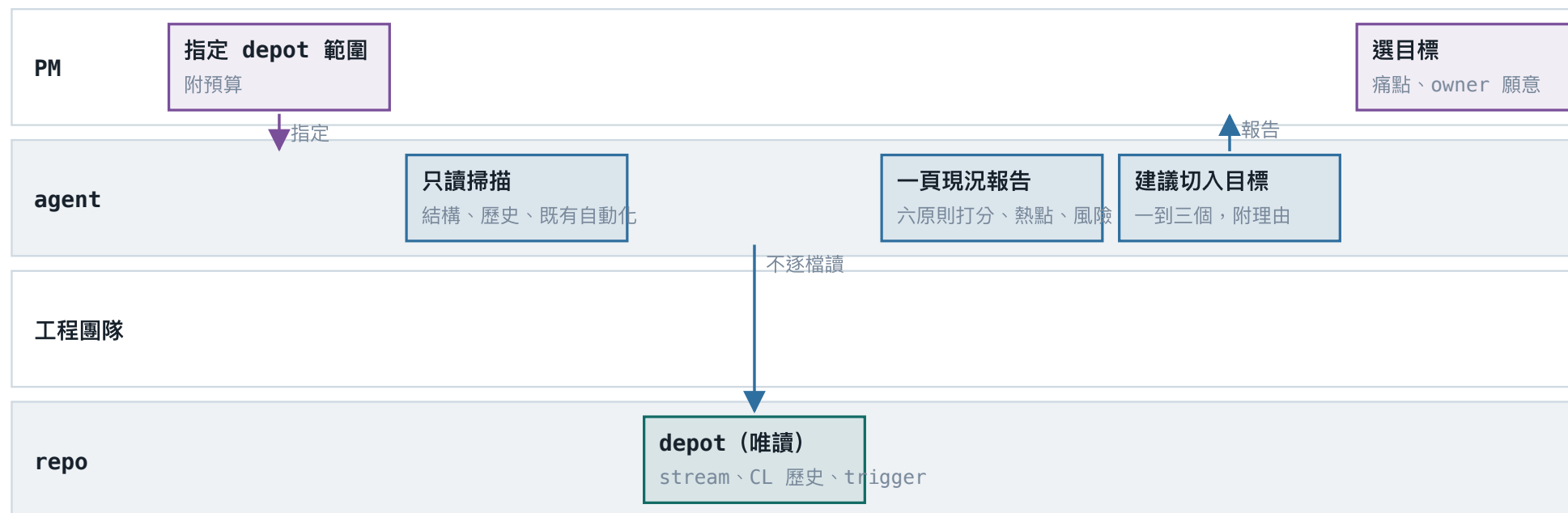
agent 用自己的 **bot** 帳號、只讀權限上線；第一件事是向團隊說自己是誰、向誰報告、會看什麼、紀錄放哪。

權限分階段：先只讀，要交 shelved CL 時加寫入，要開 stream 時再加；每一階 PM 給，agent 不自己擴。

盤點：agent 只讀掃指定的 depot 範圍，交 PM 一頁現況與幾個候選目標，PM 選

什麼時候：PM 要整體盤點，還沒決定從哪裡開始

授權：自主（只讀）



報告裡每個判斷都附依據：擷取（從檔案或歷史讀到）、實跑（在乾淨 workspace 跑過）、推測。

盤點只讀、有預算、看不完就寫清楚看了什麼沒看什麼；交出去的是一頁現況和幾個候選目標，選哪個是 PM 的事。

推測的東西（負責人、目的）一律標「推測」，等目標分析時再問本人。

起手：目標還不在版控裡，agent 把目錄整理成第一個 CL，owner 看過才 submit

什麼時候：盤點發現目標根本不在版控裡（共用磁碟、home、tarball），或全新專案的第一天

授權：讀共用磁碟、建 depot 路徑都請示



示意的第一個 CL 說明：「dma 第一次進 depot：rtl/、tb/、scripts/ 的來源共 312 檔；sim 結果與 netlist 不進（清單附在 CL）。

原目錄 /proj/chipA/dma 不動，等 owner 確認再談。」

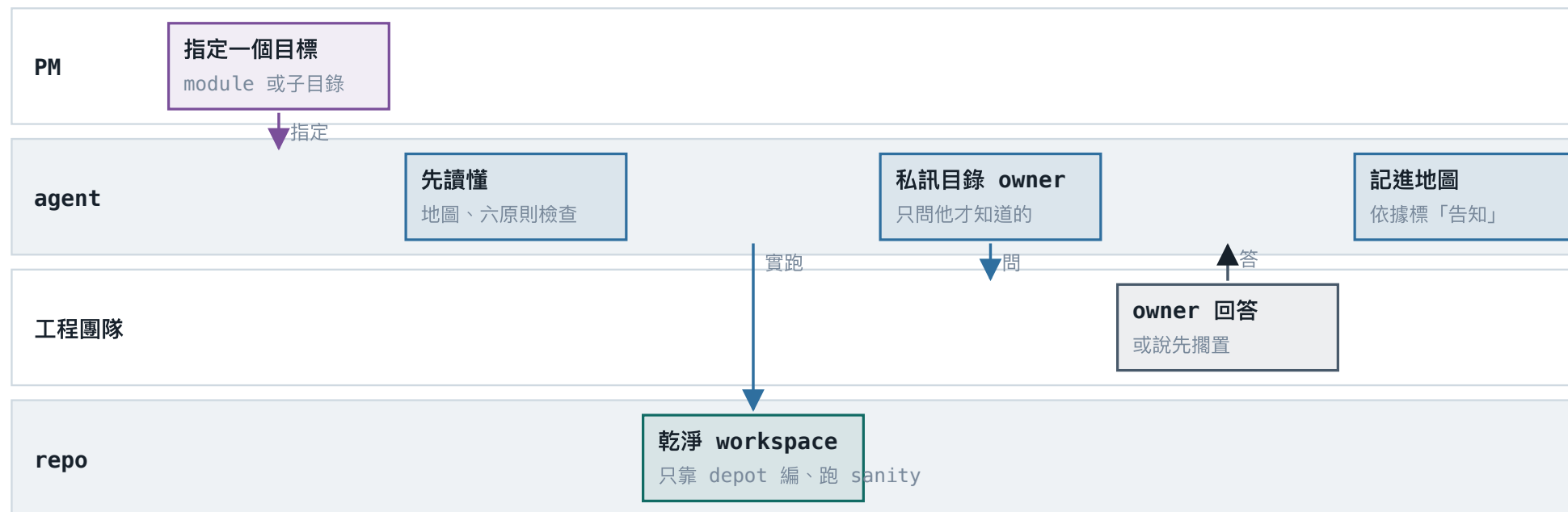
目標還沒有 repo 也接：agent 把目錄整理成第一個 CL（來源進、產物不進、附 setup 與 sanity），owner 看過才 submit；原檔不動。

全新專案更簡單：從第一個 CL 就有 sanity 與 manifest，常規從第一天教。建 depot 路徑與 stream 要 CAD，先請示。

目標分析：agent 讀完才私訊目錄 owner，只問他才知道的事，答案進 PROJECT_MAP

什麼時候：PM 指定了一個目標，agent 第一次進去

授權：自主（讀、問）



示意的私訊：「這個目錄是 DMA 的 RTL，由 tb/dma 驗，交給 top 整合，對嗎？top.f 引用的 dma_ctrl.v 我找不到，在你那裡嗎？」

只有 owner 知道的事：缺的檔在誰那裡、猜的用途對不對、交付物給誰、進行中的大包怎麼拆、flow 的正本誰維護。

問之前先查，問的時候附證據；只問只有 owner 知道的事，一次一批。

owner 說先擱置，agent 聽理由、談折衷、記下什麼時候回來問。

計畫核准：PM 核准方向、owner 同意範圍，缺一就不動手；PM 沒弄懂不算核准

什麼時候：目標分析完成，要開始動之前

授權：PM 請准 + owner 同意



計畫裡每條 patch 標三類之一：agent 自己能補、補了要 owner 決定、只有 owner 知道。

方向、優先順序、對團隊的新要求是方針層：PM 懂了才算核准，owner 同意才動他的範圍。

請准要分批、附取捨；PM 不懂就問，蓋章不算核准。

建置：骨架做成 shelved CL 交 owner，缺的 check 寫需求讓人或 subagent 做

什麼時候：計畫核准後，補某個缺口

授權：自主做；裝 trigger 請准



需求 markdown 的最低內容：目的、input、output、通過的定義、時間與資源預算、在哪裡跑、owner、怎麼測它自己。

機制分三層：觸發與流程、檢查工具、結果可見；既有的找負責人補強，沒有的用需求 markdown 讓 subagent 或人做。

不論誰做，交付都是 shelved CL 加測試證據；owner submit 才進 depot。

上線分級：check 先只報告、再警告，PM 與 owner 同意才擋 submit，留 bypass

什麼時候：一道 check 裝好之後

授權：升到擋要 PM 與 owner 請准



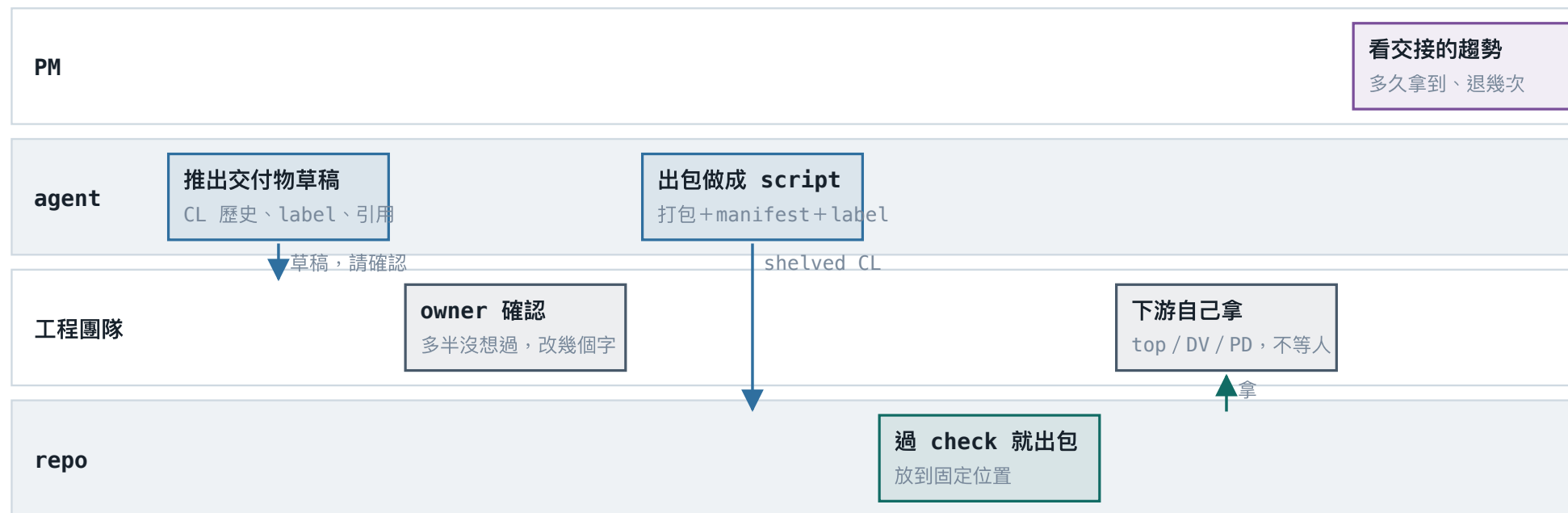
採用是階梯：只報告 → 警告 → 擋；每升一級都要證據，擋要 PM 與 owner 都同意。

沒有共識就停在警告；誤報多就退一級。

交付：agent 幫 owner 定出交付物；check 過就自動出包附 manifest，下游自己拿

什麼時候：第一道 check 穩定之後；每個目錄一次

授權：定交付物是告知；出包的 script 自主



示意的草稿：「dma 交出去的是 rtl/*.v+dma.f+manifest，給 top 整合；main 過 sanity 就打 label 放 //depot/chipA/release/dma/。對嗎？」

owner 多半沒有「交付物」的概念：agent 推出草稿，owner 只要確認；出包做成 script，main 過 check 就自動出包附 manifest。

這是 CD 的 IC 版：交到下一棒，而且下一棒拿了就能跑。

日常監看：每筆 CL agent 讀 description 與 check，看不出目的就私訊作者並示範寫法

什麼時候：第一道 check 上線之後的常態；每一筆 submit 的 CL

授權：自主（看、私訊）



示意的私訊：「這包看起來是改 DMA 的 burst，對嗎？說明可以寫成：改了什麼 / 為什麼 / 怎麼驗，例如『加 burst 模式；為了 X；跑 t_dma_burst 過』。」

監看看的是 CL 的 description、一包幾件事、check 的結果；看不出目的就私訊 CL 的作者一句並示範怎麼寫。

同一件事不重複念；壞了先問原因，可能是刻意的。

有人開新工作：agent 察覺就先問，確認後登記狀態板、代開 stream 與 check

什麼時候：看到有人好像開了一件新工作

授權：自主（問、準備）；登記要本人同意



狀態板上每件任務：做什麼（一句）、誰、動哪些目錄、預計交什麼、在哪條 stream、到哪一步；看的是任務，不是人。

看到跡象先問，說是才登記；登記時把做完的定義指到一個跑得起來的檢查。

用 Perforce 的詞輔導：stream、workspace、shelve、copy up；東西準備好交到他手上，一步不用查手冊。

擴充與交棒：一次加一道 check，團隊能自己維護後 agent 只剩監看

什麼時候：機制跑穩之後

授權：加一道告知；交棒 PM 決定



四個指標的 IC 版：submit 頻率、改動到進 main 的時間、check 失敗率、壞掉到修好的時間；以模組為單位。

一次只加一道，穩定了才加；量的是模組的趨勢，不是個人。

團隊自己維護 pipeline 之後，agent 只剩監看與維運；這就是交棒。

橫跨全程：log 與狀態板在 agent 的 repo、團隊可讀；PM 看摘要，每人可查自己的

橫跨全程：一直都在

授權：自主



agent 的日誌與狀態板放在它自己的 repo，PM 和工程師都讀得到；要副本進 depot，owner 說了算；PM 平時看一頁摘要。

關於某人的紀錄那個人看得到；報告以模組和流程為單位，不排名個人。

橫跨全程：延後可以談，被拒絕由 PM 裁決，agent 錯了公開更正

橫跨全程：對方不想動、不同意，或 agent 錯了

授權：延後可自己答應；被拒由 PM 裁決



延後：嚴重度低、有期限的 agent 自己答應；嚴重度高、沒期限、一再延後的由 PM 裁決，而且對當事人透明。

工程師要延後可以談：聽理由、談折衷、記下什麼時候回來問；被拒絕就由 PM 裁決，agent 不裁決。

agent 錯了在同一個 channel 更正，寫一頁不究責的檢討。